Project Memento: Technical Documentation & Hackathon Specification

Executive Summary

Memento is an AI-driven memory preservation and contextual biographical story generator. By combining multimodal input processing (audio interviews, legacy photographs, scanned documents) with retrieval-augmented generation (RAG) and structured timeline construction, Memento enables families to preserve, organize, and interact with personal histories across generations.

---

1. System Architecture

1.1 High-Level Component Topology

[Frontend: Next.js / Tailwind]



[API Gateway / Orchestration Layer: FastAPI]

├── Auth & Storage Engine (Firebase / AWS S3)

├── Processing Pipeline (LangChain / LlamaIndex)

└── Vector Engine (Pinecone / Qdrant)



[Model Layer: Gemini 1.5 Pro / Flash]

├── Audio Transcription (Whisper API / Gemini Multimodal)

├── Image Parsing & OCR (Gemini Vision)

└── Structural Narrative Generation (Gemini 1.5 Pro)

1.2 Data Flow Pipeline

1. **Ingestion:** Raw audio files, images, and text notes are uploaded via the Next.js web application.
2. **Preprocessing & Embeddings:**
    - Audio is transcribed with timecode alignment.
    - Visual assets undergo OCR and detailed scene/entity description.
    - Text segments are embedded and indexed in vector storage with temporal and entity metadata.
3. **Timeline & Knowledge Graph Construction:** Extracted metadata populates a structured chronological memory graph.
4. **Synthesis Engine:** User queries or context prompts trigger dynamic context retrieval to produce narrative chapters, interactive chat experiences, or synthesized audiobooks.

---

2. Core System Prompts

2.1 Narrative Extraction & Memory Structured Output Prompt

System Role: Senior Biographer and Data Structuring Specialist

Task: Analyze the provided transcript segment and extract key memories, entities, and temporal anchor points into a strict JSON payload.

Input Transcript:

"{TRANSCRIPT_TEXT}"

Extraction Requirements:

- Date/Timeframe: Exact year, decade, or approximate age if explicit.

- Core Entities: People, places, and significant artifacts.

- Key Theme: Emotional tone and overarching topic.

- Summary: A 2-3 sentence clear representation of the story.

Output Format: JSON strictly adhering to schema.

2.2 Conversational Memory Retrieval Prompt

System Role: Interactive Memory Assistant (Persona: Empathetic Family Historian)

Task: Respond to the family member's query using strictly the provided context fragments.

Context Fragments:

{RETRIEVED_CONTEXT_BLOCKS}

User Query: "{USER_QUERY}"

Guidelines:

1. Maintain a respectful, warm, and authentic conversational style.

2. Rely only on facts present in the Context Fragments. Do not hallucinate life events.

3. If information is missing or ambiguous, state this gently and suggest a follow-up interview question to ask the primary source.

---

3. Hackathon Execution Plan (48-Hour Timeline)

Phase	Duration	Core Deliverables	Tech Stack / Tools
Phase 1: Setup & Data Core	Hours 00–12	Environment configuration, vector database schema, audio processing pipeline setup.	FastAPI, Pinecone, Gemini API, Next.js
Phase 2: RAG & Synthesis	Hours 12–28	Temporal retrieval implementation, structured narrative generator, timeline frontend UI.	LangChain, TailwindCSS, React Flow
Phase 3: Interactive Polish	Hours 28–40	Conversational interface, visual memory cards, export pipeline (PDF/Markdown generation).	Next.js, Framer Motion, Vercel
Phase 4: QA & Pitch	Hours 40–48	End-to-end integration test, sample dataset population, video demonstration recording.	Loom, Vercel Deployment

---

4. GitHub README Template

Project Structure Overview

memento/

├── apps/

│ ├── web/ # Next.js frontend application

│ └── api/ # FastAPI orchestration engine

├── packages/

```
│   ├── core/            # Shared data models and types
│   └── prompts/         # System prompt templates
├── docs/                # Architectural diagrams and specs
└── README.md
```

Quickstart Guide

Prerequisites

- Node.js >= 18.x
- Python >= 3.10
- API Keys: Gemini API, Pinecone, Firebase

Installation Steps

1. Clone the repository:
```
git clone https://github.com/organization/memento.git

cd memento
```

2. Setup backend API:
```
cd apps/api

python -m venv venv

source venv/bin/activate  # On Windows: venv\Scripts\activate

pip install -r requirements.txt

cp .env.example .env

uvicorn main:app --reload
```

3. Setup frontend client:
```
cd ../web

npm install

cp .env.example .env.local
```

```
npm run dev
```

4. Access application at `http://localhost:3000`.

---

5. Metadata Sign-off

- Prepared for: Hackathon Judging Panel
- System Version: 1.0.0-beta
- Status: Complete Technical Specification